

Lab 0, Digital Signal Processing

Prof. Naomi Harte

September 10, 2026

In this optional preparatory lab, you will complete a sequence of tasks which explore some of the key undertakings of Digital Signal Processing. We provide MATLAB code snippets for many questions. These are NOT the answers to the question, but you can get inspiration from the provided syntax to derive the necessary code. A fundamental knowledge of MATLAB is required to complete these labs. You should take a refresher course online in MATLAB if you feel you need it.

Visit <https://matlabacademy.mathworks.com/details/matlab-onramp/gettingstarted> if you need a course.

Similar to the refresher MATLAB course, both parts of this lab are arranged as sequential tasks. Each task must be completed before moving onto the subsequent task. You are presented with a skeleton MATLAB file 'Lab0Part1Student.m' which you can edit to complete what is being asked of you in the Lab and keep a record of your work. A similar file will be provided for future Labs in this module. Where reference to a report is made, this is just for your own records and the report does NOT need to be submitted. If you choose to attend the Clinic for support, you should have all this material to hand. Where questions are asked, these are intended for your own reflection to test you understand the tasks.

You will need headphones to listen to the signals when necessary. Before listening to any signals, please check your volume to avoid damaging your hearing.

Remember you should complete most of this lab in ADVANCE of attending the clinic, using the clinic to get help with any remaining issues you have.

1 Part 1: Operations on Simple Sequences

To begin, please open the Matlab script 'lab0Part1Student.m' provided to you. Save the script again replacing the word 'Student' with your own name. Fill in your answer for each task in the blank section

under each question in the script.

The goal for these initial exercises is to manipulate simple sequences in MATLAB. Specifically, we consider plotting, delaying, adding/subtracting finite sequences.

Task 1 - Creating a Discrete Time Sequence or Signal.

The signal x is given as:

$$x = \delta[n] + 2\delta[n - 1] - 2\delta[n - 2] - \delta[n - 4] + 2\delta[n - 5]$$

In matlab, create an array named x which describes this signal. Refer to Lecture 2 if this notation seems unfamiliar to you.

Task 2 - Creating a Time Sequence to Match Your Discrete Signal.

Matlab presumes that the first element in an array is $n = 1$. Your sequence begins with $n = 0$.

Create a variable, t which defines the values of n associated with each element in signal x .

You can do this by defining each value in the array or by using a step size to go from the first to the last value.

Task 3 - Plotting Your Signal Against Time Using a Stem Plot.

Plot x as a function of t using a stem plot. Have you done this correctly?

```
figure % new figure
stem(t, x) % x as a function of t
```

Task 4 - Giving Your Plot a Title and Labelling Your Axes.

Give your plot a title that contains your student id, and label both the x and y axes.

```
title('my title is...') % Figure title
xlabel('the label for x axis is...')
ylabel('the label for y axis is...')
```

Include the plot in your report.

Task 5 - Controlling the Displayed Axes Limits.

Use *xlim* and *ylim* to expand the plot to show one more than the maximum values of *t* and *x* and one less than the minimum values of *t* and *x*.

```
xlim([0 1]) % Restrict x values between 0 and 1
ylim([-1 1]) % Restrict y values between -1 and 1
```

Task 6 - Delaying Your Signal.

Create a new array, *xDelayed* which is the same signal as *x*, delayed by 2 samples. This can be done using the *zeros* function and *horzcat*.

```
t = -2:step:10 %%create a vector starting at -2 and finishing at 10 with an
    increment of step
horzcat(x,y) %%concatenate two arrays horizontally
zeros(1,3) %% create an array of 3 zeros
```

Task 7 - Plotting the Delayed Signal against the Original Signal.

Plot the original signal, *x* and the delayed signal, *xDelayed* on the same figure by using the *hold* function after plotting the first signal.

```
figure %creates a new figure
stem(t,x)
hold on % ensures that whatever is plotted next appears in the same figure
stem(t,xDelayed)
```

Remember that *xDelayed* is of a different length to *t* and *x*. So *t* and *x* will need to be redefined in order to plot them all on the same figure. Pay attention to how to get this correct!

Task 8 - Labelling The Signals in Your Figure.

Like in Task 4, give your plot a title including your student id, and label the *x* and *y* axes. In addition to this, label the signals *x* and *xDelayed* using the *legend* function and specify new limits for the *x* and *y* axes so that both signals are clearly visible.

Include this plot in your report.

Task 9 - Adding Two Signals Together.

Consider 2 new signals:

$$x1 = -\delta[n - 1] + 3\delta[n - 2]$$

$$x2 = 3\delta[n - 1] + 3\delta[n - 2] + \delta[n - 3] - 2\delta[n - 4]$$

Add these two signals together to make $x_{Addition}$. Note that they must be added in a time-aligned manner. (Sketch the two sequences out for yourself to understand this point, if it is not immediately clear)

Task 10 - Plotting Three Signals in One Figure Using Sub Plot.

Using subplot, make a figure with 3 separate plots, one for $x1$, $x2$, and $x_{Addition}$.

```
figure %creates a new figure
subplot(2,1,1) %This figure will have 2 rows, 1 column, and we're selecting the
               first plot
stem()
```

Include this plot in your report. Are the signals correctly aligned? Explain briefly in your report your method to align the signals.

Task 11 - Defining an Impulse Response.

Consider the signal $x3$:

$$x3 = \delta[n + 3] - \delta[n + 1] + \delta[n - 1] - \delta[n - 3]$$

The impulse response, $h_0[n]$ is given by:

$$h_0[n] = \begin{cases} 1, & \text{if } n = 0 \\ 0, & \text{otherwise} \end{cases}$$

In Matlab, define the array $x3$ as well as $h_0[n]$ using the *UnitImpulseSeq* function.

Task 12 - Convolving a Signal With an Impulse Response.

Convolution is a mathematical way of combining two signals to form a third signal. In linear systems, it relates the input signal $x[n]$, the output signal $y[n]$, and the impulse response $h[n]$. If the system being considered is a filter, the impulse response is called the filter kernel, the convolution kernel, or simply, the kernel.

For discrete signals in general, the convolution is defined as:

$$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k]$$

Now, a new array $y[n]$ is defined in this task as the convolution of $x[n]$ and $h_0[n]$ (just generated in the task above) and is given by:

$$y[n] = x[n] * h_0[n]$$

in Matlab, generate $y[n]$ using the `conv` function. Plot the signal to study it.

What do you notice about $y[n]$? Discuss your findings in your report.

Task 13 - Plotting a Convolved Signal.

Plot both $x_3[n]$ and $y[n]$.

Include these plots in your reports. Please explain the effect of $h_1[n]$ on the input signal ($h_a[n] = 1$ for $n = a$, 0 otherwise).

2 Part 2: Sine Wave Model

We consider the following continuous signal:

$$x(t) = \sin(2\pi f_0 t)$$

It is a pure sinusoid with frequency f_0 , amplitude 1, & phase 0. The signal corresponds to a music note. The higher the frequency is, the higher the pitch of the sound. For instance, a sinusoid with frequency $f_0 = 440$ Hz corresponds to the note A.

This analogue signal cannot be studied in MATLAB as it is continuous and not discrete. We must therefore consider the digital version of this signal which is sampled and quantized.

We define the signal $x[n]$ sampled with sampling frequency f_s , which contains N samples, as:

$$x[n] = \sin(2\pi f_0 t[n])$$

The times $t[n]$ are defined as:

$$t[n] = \frac{n}{f_s} \quad \text{for } 0 \leq n \leq N - 1$$

By default, MATLAB quantizes all values with 64 bits. In this lab, the considered signal has the following features: sampling frequency $f_s = 8000$ Hz, fundamental frequency $f_0 = 800$ Hz, duration $d = 2$ secs.

To begin, please open the Matlab script 'lab0Part2Student.m' provided to you. Save the script again replacing the word 'Student' with your own name. Fill in your answer for each task in the blank section under each question in the script

Task 1 - Define your variables.

Enter the values of f_s , f_0 , and d into MATLAB.

Task 2 - Determine the number of Samples, N.

Using d , and F_s , determine the number of samples N in the sequence and define N in Matlab.

How do you choose N when d is not an integer, so that you do not lose information? Hint: there is no

problem to append zeros at the end of the signal.

Task 3 - Create a time vector, t .

Construct in MATLAB the time vector t of size N that contains the different values $t[n]$. Warning: in this lab, we work with column vectors!

Task 4 - Construct the signal x .

Construct the signal x containing the values $x[n]$ (the signal which is the sampled version of $x(t)$).

Task 5 - Listen to x .

Using headphones, listen to the signal x . Would you describe the sound as deep, high-pitched or neither? Adjust the value of f_0 between 300 and 1200 Hz. What can you infer about the physical meaning of frequency when listening for different values of f_0 ?

```
sound(x, fs) %plays the audio signal, x at a sampling rate of fs
```

Task 6 - Find T_0 .

What is the fundamental period T_0 of the signal $x(t)$ before sampling? *Include your answer in your report.*

Task 7 - Plot $x[n]$.

Plot the sampled signal $x[n]$ in the interval $[0, 4T_0]$ and annotate the figure. Try for different values of f_0 between 300 and 1200 Hz.

Please include one figure, complete with annotation in your report.

Task 8 - Delay the signal.

Delay the signal $x[n]$ by 0.5 seconds. We recall that the unit of f_s is samples/sec. Hint: you need to convert delay in seconds to delay in samples. You can use the `delaySeq` function in Matlab to do this.

```
Y = delayseq(X, DELAY, Fs) %specifies DELAY in seconds. Fs is the sampling  
frequency (in Hz).
```

Task 9 - Add to the signal, x .

Add to the signal x another signal, y_1 with fundamental frequency $f_0 = 400$ Hz, and sampling frequency $f_s = 8000$ Hz, and duration $d = 2$ seconds.

Plot the resulting sinusoid in the interval $[0, 4T_0]$ and include the plot in your report. What can you say about the signal shown over this particular period?

Task 10 - Multiply two signals.

Multiply the signal x with another signal y_2 with fundamental frequency $f_0 = 400$ Hz, and sampling

frequency $f_s = 8000$ Hz, and duration $d = 4$ seconds.

In this scenario, x and y_2 start at the same time.

Remember that when adding or multiplying sequences, they must be of the same length.

Plot the resulting sinusoid and include the plot in your report. Choose an appropriate range using `xlim` to show the sinusoid clearly.

Task 11 - Convolve two signals.

Convolve the signal x with another signal y_3 with fundamental frequency $f_0 = 400$ Hz, and sampling frequency $f_s = 8000$ Hz, and duration $d = 2$ seconds.

In this scenario, x starts at $t = 0$ and y_3 starts at $t = 1$.

Plot the resulting sinusoid and include the plot in your report. Choose an appropriate range using `xlim` to show the sinusoid clearly.

3 Part 3: Extra Optional Challenge

The following challenge will not count towards your grade for this assignment. However, successful completion of this challenge will result in your name being entered into a draw to win a prize at the end of the semester.

The .csv file IRL DLY RR 2021 grid.csv contains rainfall data from Met Eireann (The Irish Meteorological Service). This particular file contains values on a 1x1km grid Irish Grid (TM65) covering the land of Ireland for the year 2021. If you open the file (in excel or similar) you can see headers for the date and grid references. Interpolation models were used to calculate the rainfall and temperature grid point values with model inputs from observations recorded at Met Eireann stations. The daily rainfall total is presented without a decimal point. For example, a value of 123 is equivalent to a daily rainfall total of 12.3 mm. You need to isolate the annual rainfall from a single grid point and use that as one sequence. Plot the sequence to check you have isolated it correctly (sanity check - how many days are in a year? Are your units on the rainfall reasonable?) Then generate another sequence $h[n]$ which when convolved with the rainfall data sequence, will behave as a moving average filter (refer to lecture notes). Convolve the two sequences and plot the output. Compare it to the original rainfall sequence. What can you observe? Experiment with longer and shorter lengths for the $h[n]$. What can you conclude about the weights of $h[n]$ in order to truly calculate a meaningful average value? In your write up, show a few suitably annotated plots with the original sequence of rainfall, the $h[n]$ used, and the convolution output.

```
T = readtable('myfilename.csv'); %built in matlab function to read the csv
    file including headers
Tarray = table2array(T); % strip headers
s = Tarray(:,227); % row or column?
s = Tarray(227,:); % row or column?
s = Tarray(117,3:end); % skip first two elements y = conv(x,h) % convolve two
sequences
```

4 Write-up?

This lab has been provided to allow you brush-up on your MATLAB skills. Not write-up or submission is required.

5 Appendix

Below is some example code which shows how you might plot a signal in MATLAB. The figure it creates can be seen below.

Listing 1: MATLAB code to plot a signal

```
x = [1, 0, -1, 2];

t = [0:1:3];

figure % new figure
stem(t,x) % x as a function of t
title('my title is...') % Figure title xlabel('the label for x axis is...')
ylabel('the label for y axis is...')
title('this is a title')
xlim([0 4]) % Restrict x values between 0 and 4
ylim([-1 2]) % Restrict y values between -1 and 2
```

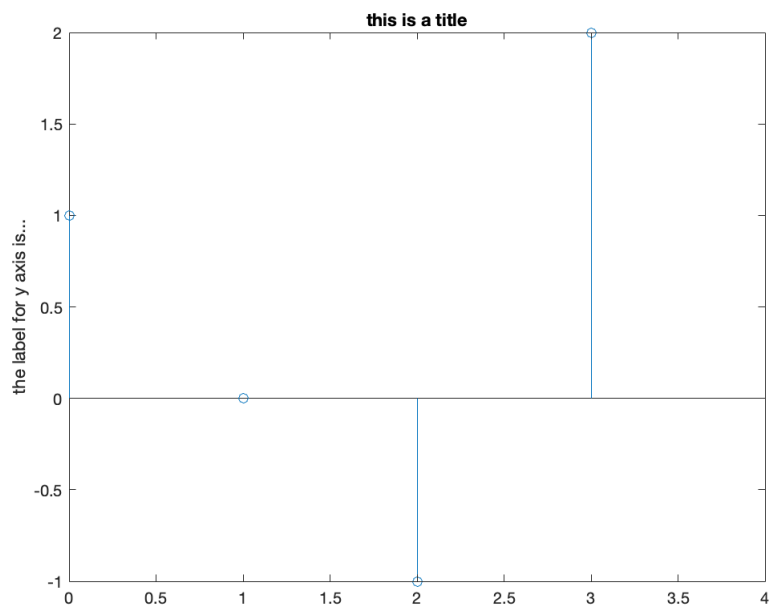


Figure 1: A MATLAB Plot of a Sequence